

Convergence Is Not Correctness

The sequential-specification campaign: independent, kernel-checked specs
for every replicated data type in Sal

Sal / FP Launchpad (KC + Claude)

2026-07-16

Abstract

Replication-aware linearizability certifies that every version of a replicated data type is the fold of some delivery-respecting linearization of its events. The reference point of that guarantee is the datatype’s *own* fold: a bug in `do` appears identically on every replica, converges perfectly, and passes every convergence theorem. This note records the campaign that closes the gap: for every production RDT in Sal, a kernel-checked theorem that the fold of any single-replica history equals the output of a straightforward sequential program, the one a programmer would write with no replication in mind, discovered through an explicit inductive invariant. Four tiers, easiest to hardest: eleven flat datatypes, the mergeable queue against a plain functional queue, the embedded-chain RGA against the naive insert-at-index text buffer, and fused Peritext against a naive marked-text editor. The campaign’s negative results carry as much information as its theorems: the rehoming tombstone-free RGA is *refuted* at this specification on a four-op single-replica trace, its fused Peritext inherits the refutation at the render, and Shesha passes sequentially while failing at the merge; together they witness that do-faithfulness and merge-faithfulness are independent axes. A final section states honestly what the campaign does *not* give: per-replica soundness and convergence do not compose into “the merged state is some user’s sequential result,” and the precise corner where composition fails is the same dead-anchor corner the sequence-CRDT trilemma names. Every theorem is kernel-clean; every claim names its Lean identifier.

1 The specification gap

The framework’s central guarantee is RA-linearizability at every reachable configuration: each version the store registers is $\text{fold}(\rho)$ for some enumeration ρ of the version’s event set that respects delivery order. This is a strong convergence guarantee and it is the right one for replication: it says the merge invented nothing, reordered nothing it was not entitled to reorder, and that all replicas agree.

It says nothing about whether the fold is *right*. The reference sequence is the datatype’s own `do`; if `do` is wrong, every replica is wrong in unison, and RA-linearizability holds vacuously over the shared mistake. Two incidents made this concrete in this repository. First, a mark-extent defect in an early Peritext render passed every convergence obligation and was caught only by eye. Second, and decisively: the rehoming tombstone-free RGA carries a fully general, kernel-clean RA-linearizability capstone (`rga_ra_linearizable3_eq`), and its `do` nonetheless reorders survivors when an interior node is deleted, with no concurrency anywhere in the history (§4). Both defects are invisible to convergence because convergence is self-referential.

The remedy is an independent specification. For sequential behaviour there is an obvious and canonical one: *the sequential program a programmer would write for the same operations with no*

replication in mind. A counter is addition. A queue is a list. A text buffer is insert-at-position and delete. A marked-text editor is a buffer of characters and boundary tokens with one left-to-right walk. The campaign’s obligation, per datatype:

$$\begin{aligned} & \text{for every sequentially honest single-replica history } \rho: \\ & \text{read}(\text{fold}(\rho)) = \text{specProgram}(\rho) \end{aligned}$$

proved by discovering an inductive invariant of the fold, never by weakening the spec to match the implementation. “Sequentially honest” is the datatype’s own issuing discipline specialized to a linear history (fresh monotone stamps, each op applicable at the state it executes against), so the theorems attack no strawman: the histories quantified over are exactly the ones a single well-behaved replica produces.

2 The four tiers and their invariants

The campaign ran easiest to hardest, and the difficulty gradient turned out to be a gradient of *invariant kinds*.

Tier 1: the flat datatypes (eleven RDTs). `SeqSpec.Flat.lean`. For most flat datatypes the fold is a view homomorphism and the invariant is trivial: the counter and PN-counter fold to arithmetic (`ioc_seq_sound`, `pn_seq_sound`), the grow-only set and map to naive insertion (`goset_seq_sound`, `gomap_seq_sound`), the OR-set and its efficient variant to a membership view (`orset_seq_sound`, `orsete_seq_sound`), the enable-wins flag and conditional G-set likewise (`ewflag_seq_sound`, `gsetcond_seq_sound`), the LWW and FWW registers to last- and first-write selection (`lww_seq_sound` via the stamp-order invariant `lww_fold_lt`, `fww_seq_sound`). Two datatypes need genuine *identity-hygiene* invariants, facts about tag bookkeeping that hold on every reachable single-replica state and are exactly what the induction needs: the multi-valued register’s overwritten tags are always a subset of previously staked tags (`mvr_over_tags`, feeding `mvr_seq_sound`), and the add-wins priority queue’s stake tags stay duplicate-free (`awpq_mem_seq_sound`, `awpq_inc_log_sound`).

Tier 2: the mergeable queue = a plain FIFO. `MergeableQueue_SeqSpec.lean`: `queue_seq_sound`, with the identity-hygiene invariant `q_tags_nodup`. Notably lean: its axiom audit is `{propext, Quot.sound}`, with no choice anywhere.

Tier 3: the embedded-chain RGA = the naive text buffer. `EmbedRGA_SeqSpec.lean`: `embed_seq_sound`. Here the invariant becomes *geometric*. The spec program splices an insert immediately after its anchor; the datatype sorts records by birth-chain coordinates. The bridge is the **adjacency lemma** (`chainBefore_snoc_iff`): a fresh insert’s chain $c_a \mathrel{++} [\delta]$ sits directly after its anchor’s chain c_a in display order, because the fresh timestamp gap δ exceeds every delta any existing chain hangs off c_a , a fact that costs only sum-telescoping over the chain (every chain’s deltas sum to its id, and every existing id is below the fresh stamp). The sorted canonical state then *is* the spec buffer, record for record. Because the published tombstoned RGA is read-equivalent to the embed on honest event sets (the compaction theorem), it inherits the buffer spec as a corollary: `rga_seq_read_eq_buffer` and the element-level `rga_seq_read_element` (`EmbedRGA_SeqSpec_RGA.lean`, a standalone build target because the two RGA models share top-level names). And because the sided generalization simulates the one-sided instance on all-R

histories unconditionally (`sFold_liftOp`), the buffer spec transports to the sided datatype’s all-R fragment at zero proof cost.

Tier 4: fused Peritext = a naive marked-text editor. `PeritextEmbed_SeqSpec.lean`: `peritextEmbed_seq_sound` and the id-tagged `peritextEmbed_seq_sound_ids`. The editor is a plain buffer of (id,element) entries, elements being characters or mark boundaries, with one left-to-right walk maintaining the open-mark set: a start boundary opens its mark, the matching close removes it, characters display with whatever is open. The theorem is a *composition*, and that is the finding: tier 3’s buffer soundness, made payload-generic, already says the fold’s (id,element) sequence is the naive buffer, and the datatype’s render and the editor’s screen are literally the same pure fold over that sequence. No new invariant was needed. This is also why the tier had to wait for the re-based Peritext (§4): against the rehoming kernel the same statement is false.

The gradient, summarized: *identity hygiene* (tags are staked, tags are nodup) for the flat and queue tiers, *geometry* (adjacency, stamp monotonicity) for the sequence tier, *composition* (buffer $\times$ shared render fold) for rich text.

3 The specifications, stated

This section writes out the sequential specification of every RDT in the campaign, and flags, per datatype, where the RDT’s operation alphabet had to *bend away* from the purely sequential data structure’s interface. That bend is the campaign’s most instructive artifact. A sequential structure addresses its state positionally or implicitly: `dequeue()` takes no argument because “the head” is unambiguous; `insert(i, e)` names an index. A replicated datatype cannot, because positions and implicit referents are not stable under concurrent edits. So the RDT operation carries a **witness**: the identity (tag, stamp, id, coordinate, snapshot) of what the issuing replica *observed* the positional referent to be, at its own state, at issue time. The sequential-honesty side conditions of §2 are then exactly the statement “the witness is what the sequential operation would have seen,” and each theorem says: under that discipline, the witnessed-identity program implements the positional program. Where no witness is needed, the op alphabets coincide and the spec is a view homomorphism.

3.1 No gap: the alphabets already coincide

For nine of the flat datatypes the RDT op is literally the sequential op; the state carries invisible bookkeeping (tags, per-replica slots) and the spec is the evident program on the user-visible view.

- **Counter / increment-only counter.** Spec state $\mathbb{Z}$ (resp. $\mathbb{N}$); `inc` is $n \mapsto n + 1$. Theorem: the fold *is* the history length (`counter_seq_sound`, `ioc_seq_sound`).
- **PN-counter.** Spec: `#inc - #dec` (`pnSpec`); `pn_seq_sound`.
- **Grow-only set / map, conditioned G-set.** Spec: accumulation; membership of x iff some op added x (`goset_seq_sound`, `gomap_seq_sound`, `gsetcond_seq_sound`).
- **OR-set (plain and efficient).** Spec state: a plain set; `add e` inserts, `rem e` deletes (`orSpecStep`). The tag machinery (fresh (t, e) stakes, filter-all-tags on remove) is invisible to one replica: the element view of the fold is the naive program (`orset_seq_sound`, `orsete_seq_sound`). No invariant needed; the view is a fold homomorphism.

- **Enable-wins flag.** Spec state: one boolean; $\text{enable} \mapsto \text{true}$, $\text{disable} \mapsto \text{false}$ (ewSpecStep); ewflag_seq_sound .
- **Add-wins priority queue.** Two-part spec pinning both state components: membership behaves as the plain add/remove set with inc inert (awpqSpecStep , $\text{awpq_mem_seq_sound}$), and the increment component is a grow-only log of exactly the issued increments ($\text{awpq_inc_log_sound}$).

3.2 Gap: arbitration by stamp instead of program order

LWW and FWW registers. The sequential register is write v with *program order* deciding who wins: the spec programs are “the last write’s value” (lwwSpecFold : $_, o \mapsto \text{some}(\text{val } o)$) and “the first write’s value” (fwwSpecFold). The RDT op carries its Lamport stamp *as the arbitration key in the payload* (a max- resp. min-semilattice on stamped writes), because concurrent replicas have no shared program order to appeal to. The gap is the transfer of ordering authority from control flow to data. The honesty condition is precisely its repair: **stampsMono** (stamps strictly increase along the history, the sequential Lamport condition), under which the arbitration key agrees with program order and lww_seq_sound / fww_seq_sound recover last-write and first-write semantics. Without **stampsMono** the statements are false and should be: a replica that back-dates a write is asking LWW to disagree with program order.

3.3 Gap: overwrite by snapshot instead of “everything”

Multi-valued register. The sequential register’s write v implicitly overwrites *everything before it*. The RDT op is $\text{write}(v, O)$ where O is a **prepare-time snapshot**: the set of write-stamps the issuer currently sees, named explicitly so that a *concurrent* write (not in O) survives, which is the whole point of an MVR. The sequential spec is still the plain last-write register (mvrSpecFold). Honesty (mvrOK): the op’s stamp is fresh and O lists *exactly* the issuer’s visible stamps, the formal form of “I overwrite precisely what I can see.” Under it, mvr_seq_sound : the visible-value view of the fold is the last write. The proof needs the campaign’s first genuinely discovered invariant, mvr_over_tags : every overwritten stamp was staked, which is what makes a fresh stamp provably not-yet-overwritten.

3.4 Gap: dequeue() becomes delete-what-I-saw

The mergeable queue, the cleanest specimen of the principle. The sequential queue is

$$\text{enq } v : S \mapsto S \mathbin{++} [v] \qquad \text{deq}() : S \mapsto \text{tail}(S)$$

with deq *nullary*: the head is implicit. The RDT cannot say “the head” (two replicas may disagree about it), so its op is $\text{deq } t$: remove *by tag*, where t is the tag of the element the issuing replica read as the head of *its own* queue. The implementation filters t out of the whole list, wherever it sits. The spec keeps the sequential shape, tail , blind to t (qSpecStep); honesty (qOK) says the witness is faithful: the fold at the issue point is literally $(t, v) :: \text{rest}$. The invariant q_tags_nodup (fresh stakes stay duplicate-free) makes filter-by-tag remove exactly the head, and queue_seq_sound closes the loop: delete-what-I-saw implements pop-the-head. The same witness discipline is what the convergence side already consumed ($\text{qHonest_of_applicable}$); the campaign reuses the datatype’s own contract rather than inventing one.

3.5 Gap: insert-at-index becomes insert-after-who-I-saw

The embedded-chain RGA. The sequential text buffer is $\text{insert}(i, e)$ and $\text{delete}(i)$, index-addressed. The RDT op is $\text{ins}(e, \pi, a)$: a is the *id* of the element the issuer saw at position $i - 1$ (the anchor; the sentinel 0 for the head), and π is the anchor’s stored coordinate as the issuer read it (the proof-carrying prefix); $\text{del}(x)$ names the id seen at position i . The spec program is the buffer in id-clothing (**eSpecStep**): front-cons for $a = 0$, otherwise splice immediately after a ’s entry (**eInsAfter**); delete filters by id. On a sequential history the anchor id determines the index uniquely, so this *is* the positional program with positions named by their occupants. Honesty (**eSeqOK**): stamps exceed all prior insert stamps, and each op is **eApplicable** at the current fold, i.e. the anchor is live and carries exactly π : the witness is faithful. The theorem (**embed_seq_sound**) is the strongest form in the campaign: the canonical sorted state equals the spec buffer *record for record*, via the adjacency lemma. The same spec, verbatim, is what the rehoming RGA is refuted against (§4): the gap analysis there is not about the witness (the rehoming op carries one too, plus a path) but about the implementation moving other elements’ referents after a delete.

3.6 Gap: one mark becomes a boundary pair

Fused Peritext. The sequential marked-text editor has an *atomic* $\text{mark}(i, j, m)$. The fused RDT has no such op: a mark is **two** ordinary inserts of boundary elements $\langle m \rangle$, $\langle /m \rangle$ sharing a fresh mark id, anchored at the range ends, and mark removal is two deletes. The atomic sequential op is traded for a pair of witnessed inserts, which is the atomicity horn of the mark-positioning trilemma, accepted knowingly. The spec therefore lives at the token level: the editor is the tier-3 buffer over $\text{char} \oplus$ boundary entries plus one left-to-right walk maintaining the open-mark set (a start boundary opens, the matching close removes, characters display with what is open). **peritextEmbed_seq_sound** says the datatype’s render is that editor’s screen; the range-level reading is recovered by the positional intent theorems (**render_id_active_iff_between**: a character carries mark m iff it sits, in reading order, strictly between m ’s boundaries). The boundary-pair encoding is also why the character/boundary distinction is load-bearing in the delete theorem: deleting a *character* re-formats nothing (**renderIds_del**), deleting a *boundary* is mark removal and must re-format its span.

The pattern, in one sentence: *every honesty condition in the campaign is the faithfulness of a witness that replaced a positional argument*, and the campaign’s theorems are the statement that witnessed operations, honestly issued, are positional operations.

4 The negative rows, and the two-axis separation

The campaign’s statement shape is falsifiable per datatype, and three refutations landed. All three are machine-checked.

The rehoming RGA fails at do. **RGA_Rehoming/RGA_SeqSpec_Refuted.lean: rehoming_seq_refuted.** The witness (Figure 1) is four ops on one replica, honest in the datatype’s own terms (every op **accurate** and **fresh_ts** at its state): build $\langle b, a, c \rangle$ with c under a , then delete a . The naive buffer keeps survivor order, $\langle b, c \rangle$; the rehoming **do** re-homes c to the root, where the newest-first sibling tiebreak leapfrogs it over b : the read is $\langle c, b \rangle$. The should-pass half (**seq_agrees_before_delete**) shows both sides agree on the delete-free prefix, isolating the delete as the exact point of departure. The design’s convergence capstone is untouched: it converges, to its own reordering fold.

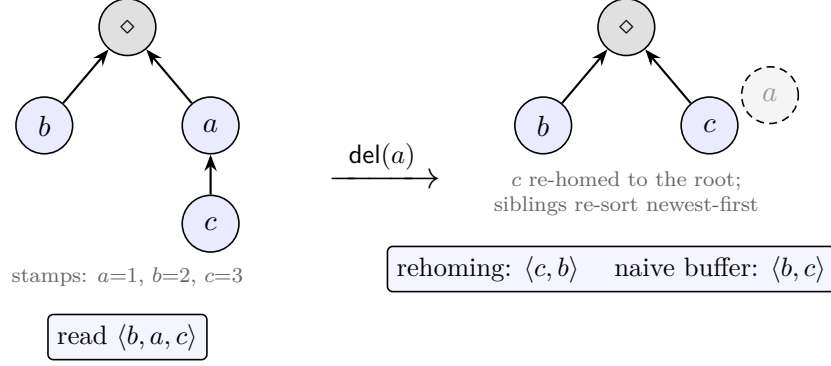


Figure 1: The four-op refutation trace. One replica, no merge. Deleting a swaps the surviving b and c in the rehoming RGA’s read; the naive text buffer keeps their order. Machine-checked as `rehoming_seq_refuted`; the embedded-chain RGA reads $\langle b, c \rangle$ on the same trace, because c ’s immutable coordinate still carries a ’s chain as pure geometry.

Fused Peritext on the rehoming kernel fails at the render. `Peritext_Rehoming/Peritext_Read.lean`: `fused_delete_reformats_survivor`. The same reorder, lifted to $\text{char} \oplus$ boundary: deleting a plain character re-homes a survivor past a mark boundary, and an untouched character changes formatting, single-replica. This is what converted the rehoming design’s demotion from taste into obligation, and what made the re-base of Peritext onto the embed kernel a correctness fix. On the embed kernel the corresponding *general* theorem holds: `renderIds_del` (axioms $\{\text{propext}, \text{Quot.sound}\}$): deleting a character leaves every other character’s rendered formatting bitwise untouched, because character entries are open-set-neutral and delete is a filter on an order-canonical state. Its boundary hypothesis is necessary and machine-checked so (`boundary_delete_breaks_filter_eq`): deleting a *boundary* is mark removal and rightly re-formats the span.

Shesha passes sequentially and fails at the merge. The mirror image. Shesha’s single-replica reads are the naive linked list (`sequential_soundness`, kernel-clean), and it uniquely preserves survivor order under delete among the tombstone-free designs of its generation (`delete_preserves_survivor_order`). Its *merge* is machine-checked non-RA-linearizable (`Shesha_Rows_Refuted`): at an honest reachable configuration the three-way merge splits a deleted node’s children around a concurrent sibling, producing a read that is the fold of no delivery-respecting linearization.

5 What composition does not give

It is tempting to compose the two guarantees: RA-linearizability says every version is $\text{fold}(\rho)$ for some delivery-respecting ρ over the event union; the campaign says folds of sequentially honest histories equal the spec program. May one conclude that every merged document is the naive program’s output on *some* ordering of everyone’s operations? **No**, and the failure is precise.

The campaign’s theorems quantify over *sequentially honest* histories: stamps monotone over inserts, every op applicable at the fold of its own prefix. An RA-linearizability witness ρ is delivery-respecting but need not be sequentially honest on either count. Stamp monotonicity fails harmlessly (any enumeration can be re-sorted when the state is a function of the event set, which is exactly fold-canonicity). Applicability is the real corner: in the union of two branches, an insert’s anchor

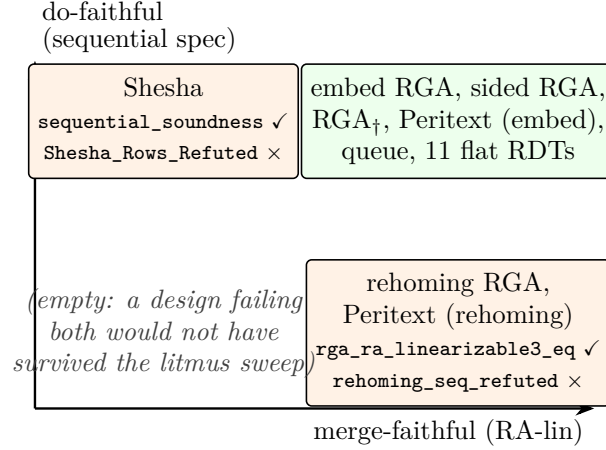


Figure 2: The two-axis separation, every placement kernel-checked. Do-faithfulness (the fold matches the naive sequential program) and merge-faithfulness (RA-linearizability) fail independently: the rehoming family converges to a reordering fold; Shesha folds correctly and merges outside every linearization. The campaign’s informative content is that the top-right corner is achievable, and by the same design idea in both sequence datatypes: immutable birth coordinates.

may be deleted by a *concurrent* op carrying a smaller Lamport stamp, so in stamp-sorted order the delete precedes the insert and the insert is not applicable at its prefix: the naive buffer no-ops it, while the datatype (whose coordinates travel in the op) places it correctly. Concretely: `ins(2 after 1)` concurrent with `del(1)`. The order `[ins1, ins2, del1]` replays to $\langle 2 \rangle$ on both sides; the order `[ins1, del1, ins2]` makes the buffer drop 2 while the fold keeps it. Some interleavings of the union match the spec program and some do not, and *whether a matching sequentially honest interleaving always exists on honest configurations is open*. We record it as such:

Open question (oq:seqwitness). On every honestly reachable configuration of the embedded-chain RGA, does the event union admit a delivery-respecting enumeration that is also sequentially honest (so that the merged read is the naive buffer’s output on it)? Caution from adjacent territory: for the rehoming design, the analogous “order dependency edges first” discipline is refuted (DepComp, via rehoming non-transitivity). The embed’s immutable coordinates remove the refutation’s mechanism, but no proof exists in either direction.

Independently of that question, the composition that *is* sound and kernel-checked today: every version is the datatype’s fold of a delivery-respecting enumeration (RA-lin), the fold is canonical per event set (`e_fold_canon`), each replica’s own contribution obeyed the sequential spec while it was being made (this campaign), and merge-level intent is delivered by separate, named theorems where it holds: delete-order preservation as sublist stability, subtree convexity as non-interleaving, read-equivalence to the published RGA as the compaction theorem. The campaign does not manufacture a merge-level user story; it pins the single-replica story so that merge-level claims have something sound to stand on.

6 The verified matrix, consolidated

The anomaly-comparison matrix (task #64, `whiteboard/anomaly-matrix/`) ran six implementations and four production libraries through 25,917 merges per row, every cell backed by executed code. Its in-repo rows have since been upgraded from “machine-run” to “kernel-checked”: the table below is the matrix’s load-bearing core with each cell a Lean theorem. External libraries (Yjs, Automerge, Loro, list-positions) remain empirical by nature; their rows live in the matrix report unchanged.

design	sequential spec (do)	RA-lin (merge)
embedded-chain RGA	<code>embed_seq_sound</code>	<code>embed_ra_linearizable3</code>
sided RGA (all-R)	via <code>sFold_lift0p</code>	<code>sided_embed_ra_linearizable3</code>
published RGA _†	<code>rga_seq_read_eq_buffer</code>	<code>rgaWithTombstones_ra_lin..._eq</code>
Peritext (embed)	<code>peritextEmbed_seq_sound</code>	<code>peritextEmbed_ra_linearizable3</code>
mergeable queue	<code>queue_seq_sound</code>	<code>queue_ra_linearizable3</code>
eleven flat RDTs	<code>*_seq_sound</code>	<code>*_ra_linearizable3_eq</code>
rehoming RGA	<code>refuted: rehomining_seq_refuted</code>	<code>rga_ra_linearizable3_eq</code>
Peritext (rehoming)	<code>refuted (render): fused_delete_reformats_surv...</code>	<code>peritext_ra_lin..._eq</code>
Shesha	<code>sequential_soundness</code>	<code>refuted: Shesha_Rows_Refuted</code>

Axiom audits: every positive theorem above is kernel-clean (axioms $\subseteq$ {`propext`, `Classical.choice`, `Quot.sound`}; the queue’s and `renderIds_del`’s audits omit choice). Refutations by concrete witness use the SPOT convention (`native_decide`, adding the compiler axioms); the refuted *statements* are quantified properties, so each refutation discharges its honesty guards on the witness and is not an artifact of a malformed trace.

7 Method notes

Three practices earned their keep and are recorded as convention. *Falsifiable statement first*: each tier states the equality before any proof work, and a refutation is a first-class outcome (three landed). *The spec is external*: the sequential program is written from the datatype’s user-facing intent, never read off the implementation; where the two disagree, the disagreement is the result. *PASS and FAIL shaped tests*: every concrete SPOT block carries should-fail companions with hand-derived expected values (the wrong answer asserted wrong), so the harness demonstrably discriminates; the campaign’s own refutation files pair each negative with the positive prefix on which the sides still agree, isolating the departing operation exactly.

Mechanization map. Tier 1: `MRDT_Instances/SeqSpec_Flat.lean`. Tier 2: `MergeableQueue/MergeableQueue_SeqSpec.lean`. Tier 3: `EmbedRGA/EmbedRGA_SeqSpec.lean` and `EmbedRGA/EmbedRGA_SeqSpec_RGA.lean` (standalone target). Tier 4: `Peritext_Embed/PeritextEmbed_SeqSpec.lean`. Negative rows: `RGA_Rehomining/RGA_SeqSpec_Refuted.lean`, `Peritext_Rehomining/Peritext_Read.lean` §10, `Shesha/Shesha_Rows_Refuted.lean`. The render fix theorem: `Peritext_Embed/PeritextEmbed.lean`. All paths relative to `Sal/ConditionedMRDTs/`.